

Bases de Données, Introduction à l'optimisation

Chapitre 03: Structures d'index - Arbre B

Asma BERRIRI

asma.berriri@universite-paris-saclay.fr

Page web du cours :

[https://sites.google.com/view/asmaberriri/home/
aspects-systeme-base-de-donnees](https://sites.google.com/view/asmaberriri/home/aspects-systeme-base-de-donnees)

Table of Contents

1 Indexation de fichiers

2 L'arbre-B

Stockage en blocs/pages

- Les données sont stockées sur un support persistant
- L'unité de stockage est : le bloc / page
- Opérations élémentaires pour accéder aux données stockées : lire un bloc, écrire un bloc
- Un enregistrement = donnée stockée = une ligne d'une table
- Stocké dans les blocs/pages d'un fichier
- Un enregistrement a un identificateur unique :
 - *ROWID* = adresse localisant un enregistrement
 - *ROWID* = (nomFichier, N° bloc, position dans le bloc)
- **Avantage du *ROWID* : accès direct au bloc contenant un enregistrement**

- Non trié :
 - Très facile à maintenir en mise à jour
 - Parcourir toutes les pages/blocs quelque soit la requête
- Trié :
 - Un peu plus difficile à maintenir
 - Parcours raccourci car on peut s'arrêter dès qu'on a les données cherchées

⇒ presque toujours un compromis à faire entre lecture et écriture

Index = concept bien connu

- Un livre à contenu technique. Il contient (au moins) un index
 - L'index présente les termes importants, classés par **ordre** alphabétique
 - À chaque terme sont associés les numéros de page où on trouve le terme
 - En parcourant l'index (par dichotomie¹) on trouve la (les) pages qui nous intéressent
- En BD: même principe ! C'est un fichier *I* permettant de trouver un enregistrement dans une table
 - ↪ **Clé d'indexation**(*k*) = une liste (l'**ordre** est important!) d'un ou plusieurs attributs
 - ↪ A chaque *k* correspond un ou plusieurs **ROWID** (adresse de bloc, enregistrement)
 - ↪ **Entrée d'index** : enregistrements de la forme [*k*, **ROWID**]
 - ↪ Un **index** est un **fichier I structuré** dont les enregistrements sont des *entrées*
 - ↪ *I* est **ordonné** sur les valeurs de *k*

¹[wikipedia] Trouver la position d'un élément dans un tableau trié: comparer l'élément avec la valeur de la case au milieu du tableau ; si les valeurs sont égales, la tâche est accomplie, sinon on recommence dans la moitié du tableau pertinente

- Un index est construit sur une clé, et sert à accélérer les recherches pour lesquelles cette clé sert de critère
- Ex. table *Film*
 - Clés d'indexation possibles:
 - Le titre du film (c'est aussi la clé primaire), l'année du film
 - Une paire constituée du titre et de l'année
 - Opérations possibles:
 - Rechercher 'Vertigo'
 - Rechercher les films parus entre 1960 et 1975
 - Rechercher les films commençant par 'V'
- L'index ne sert à rien pour toute recherche ne portant pas sur la **clé d'indexation**
 - En l'absence d'un index
 - seule solution : le **parcours séquentiel** du fichier
 - Avec un index :
 - **accès direct** à l'enregistrement
 - amélioration très importante des temps de réponse

Organisation indexée de fichiers (1/3): Clé

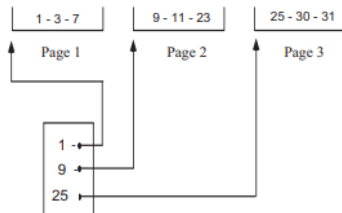
- Clé de l'index = un ou plusieurs attributs
- Stockage en **triant les données** selon la clé de l'index (Définition de l'organisation des données lors de la création de la table)
- Les enregistrements ayant la même valeur de clé:
 - ↪ sont contigus dans un bloc de donnée
 - ↪ et dans les blocs contigus suivants si nécessaire
- Index pour atteindre rapidement le bloc correspondant à la valeur de la clé (on parle d'index plaçant)
- Exemple d'une bibliothèque
 - ↪ Les ouvrages sont rangés par thème
 - ↪ On peut retrouver les ouvrages d'un thème en allant vers l'étagère de ce thème

Organisation indexée de fichiers (3/3): Index non-dense

- Objectif: obtenir un index occupant moins de place lorsque les données sont triées
- Méthode: enlever des entrées $\Rightarrow$ Ne garder que les entrées nécessaires pour atteindre le bloc de données contenant les enregistrements recherchés
 - $\hookrightarrow$ Garder une entrée de chaque bloc
 - $\hookrightarrow$ Ne pas indexer 2 fois la même clé dans 2 blocs consécutifs
- Inconvénient: toutes les valeurs de l'attribut indexé ne sont pas dans l'index
- Rmq: Un index contenant toutes les valeurs de la clé est dit dense

données

index



Organisation indexée: Index non plaçant (1/2): Définition /entrée

- Un index non plaçant est dit secondaire
- Permet d'indexer des données quelle que soit l'organisation existante du stockage
 - Données stockées sans être triées
 - Données triées selon un attribut autre que celui indexé
- Définir un index non plaçant en SQL
 - *create index Nom on NomTable (Attributs);*
 - *create index IndexAge on Personne(âge);*
- Une entrée = structure associative
 - clé → *ROWID* des enregistrements
 - Le *ROWID* permet de lire le bloc contenant l'enregistrement

Organisation indexée: Index non plaçant (2/2): Entrées

● Entrée unique

↪ Un index est dit unique si l'attribut indexé satisfait une contrainte d'unicité

- Contrainte d'intégrité dans un create table

Primary key (attribut1, ...)

Unique (attribut1, ...)

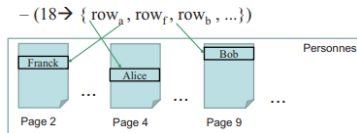
↪ Entrée d'index = (clé, *ROWID*): un seul enregistrement par valeur

● Entrée multiple

↪ Lorsque l'attribut indexé n'est pas unique

↪ Entrée d'index = (clé, liste de *ROWID*)

↪ Plusieurs enregistrements par valeur



1er exemple : Table *Film*

titre	année	...	titre	année	...
Vertigo	1958	...	Annie Hall	1977	...
Brazil	1984	...	Jurassic Park	1992	...
Twin Peaks	1990	...	Metropolis	1926	...
Underground	1995	...	Manhattan	1979	...
Easy Rider	1969	...	Reservoir Dogs	1992	...
Psychose	1960	...	Impitoyable	1992	...
Greystoke	1984	...	Casablanca	1942	...
Shining	1980	...	Smoke	1995	...

- 1 000 000 (un million) de films
- Un enregistrement = 1200 octets
- Un bloc = 4 096 octets

↪ $\lfloor \frac{4096}{1200} \rfloor = 3$ enregistrements par bloc

↪ environ $\lceil \frac{1000000}{3} \rceil = 333334$ blocs (Pour simplifier soit 300 000 blocs)

2ème exemple: Table *Cours*

Bloc	CODE	Intitule	Responsable	nb_auditeur
B1	glg102	SGBD Aspects Système	Asma Berriri	50
	glg203	Complément Objet	Frederic Voisin	100
	nfa114	Projets	Anastasia Bezerianos	30
B2	smb114	Compilation	Frederic Voisin	40
	nfe212	Intro base de données	Asma Berriri	45
	nfe006	Programmation C++	Emmanuelle Frenoux	42
B3	nfa003	Interfaces HM	Anastasia Bezerianos	37
	nfe115	Concepts Objet et Java	Aurélien Max	38
	nsy114	Vérif. and validation	Burkhart Wolff	72

- *CODE*, *nb_auditeur* codés sur 2 o chacun, *Intitule* et *Responsable* sur 60 o chacun
- entête d'un bloc de données de 18 octets
- notre SGBD constitue des blocs de 512 octets

→ Taille d'un enregistrement $(2 + 2 + 60 + 60) = 124$ octets

→ Entête bloc fait 18 o, un bloc fait 512 o. bloc utile : $512 - 18 = 494$ o

→ Nb enregistrements par bloc : $\lfloor \frac{494}{124} \rfloor = 3$ (on ne coupe pas d'enregistrement en 2)

→ Il y a 9 enregistrements, donc il faudra $\lceil \frac{\text{nbre total enregistrements}}{\text{nbre enregistrements dans un bloc}} \rceil = \lceil \frac{9}{3} \rceil = 3$ blocs

Deux étapes pour faire une recherche par index :

- ❶ Parcours de l'index (*IndexScan*)
 - Traversée de l'index
 - Retour : liste de *ROWID* (adresse d'un enregistrement-bloc)^a correspondant
- ❷ Accès direct (*DirectAccess BY ROWID*) :
 - Lecture du bloc à l'adresse (*ROWID*)
 - Parcours séquentiel du bloc pour retrouver la(les) valeur(s) de *k*

^aNotation uniquement pour notre cours:

ROWID $\equiv$ *adresse-bloc-sur-disque@position-dans-bloc*

(Ex.: le *ROWID* du 3ème enregistrement dans le 2ème bloc est noté: 2@3)

Index non-dense

- Le **fichier de données** est **trié sur la clé k**
- L'index ne référence que la **première valeur de chaque bloc** (page)
- Une page/bloc = un pointeur
 - ↳ Valeurs ordonnées physiquement sur la clé k
 - Une clé de l'index = premier tuple/enregistrement de chaque page/bloc
 - Un seul ordonnancement = un seul index non dense par fichier de données

Exemple: Dictionnaire

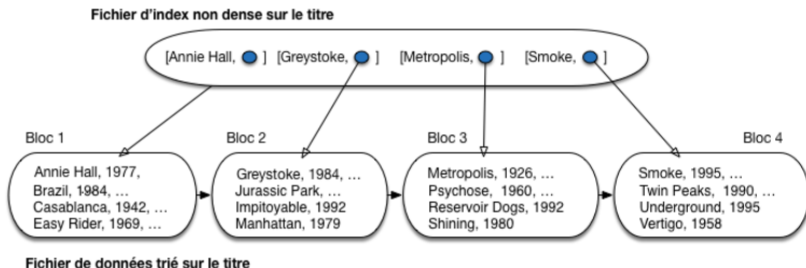
- Un index qui ne référence que le premier mot de chaque page
- Trouver n'importe quel mot ("armée", "crabe", "botte", "belle")
- Recherche d'un mot m en deux étapes. Grâce à l'index je trouve la page p associée au mot précédant immédiatement m ; puis j'accède à la page p et je cherche le mot par une recherche dichotomique locale

...

ballon,	page 56
bille,	page 57
bulle,	page 65
cable,	page 72

...

Index non-dense, exemple



- ⇒ Un fichier de 16 films. La clé est le *titre* du film
- ⇒ Chaque bloc de données contient 4 enregistrements $\Rightarrow \lceil \frac{16}{4} \rceil = 4$ blocs de données
- ⇒ Il suffit alors de 4 entrées [*titre*, *ROWID*] pour indexer le fichier

- Recherche d'articles/enregistrements de clé k :

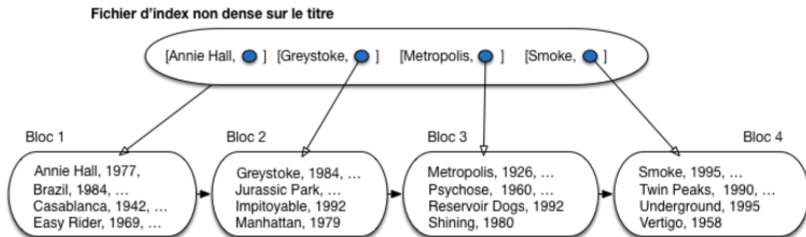
- 1 On cherche dans l'index la plus grande clé k' tel que $k' < k$
- 2 On lit le bloc données qui correspond à k' et on cherche les articles de clé k dans ce bloc (et éventuellement dans les blocs suivants)

- Par intervalle :

- 1 Rechercher par clé k l'adresse de l'enregistrement correspondant à la borne *inf* de l'intervalle. Accéder au fichier grâce à cette adresse
- 2 Partir de cet emplacement et effectuer un *parcours séquentiel* pour obtenir tous les enregistrements cherchés
- 3 Arrêter quand on trouve un enregistrement dont la clé est supérieure à la borne *sup* de l'intervalle

- Par préfixe : Oui
- Mais *pas par suffixe* : NON

Index non-dense, exemple de recherche



Fichier de données trié sur le titre

↪ Je recherche $k = \text{'Shining'}$:

- cherche dans l'index la plus grande k' tel que $k' < \text{'Shining'} \Rightarrow k' = \text{'Metropolis'}$
- lit Bloc-3 qui correspond à k' et cherche les articles de clé $k = \text{'Shining'}$ dans ce bloc (et éventuellement dans les blocs suivants)

↪ Tous les films dont le titre commence par une lettre entre "J" et "P"

- recherche dans l'index la plus grande valeur $< \text{'J'}$: c'est 'Greystoke' ;
- accède au Bloc-2, et on y trouve le premier enregistrement avec un titre commençant par "J", soit 'Jurassic Park' ;
- parcourt la suite du fichier jusqu'à trouver 'Reservoir Dogs' qui est au-delà de l'intervalle de recherche: tous les enregistrements trouvés durant ce parcours constituent le résultat de la requête

- Sur notre fichier de un million de films
 - Supposant qu'un *titre* occupe 20 octets, une adresse 8 octets
 - Taille de l'index : $300000 * (20 + 8) = 8,4 \text{ Mo}$ (à comparer aux $\approx 1,3 \text{ Go}$ du fichier de données)
 - les enregistrements dans l'index sont beaucoup plus petits que ceux du fichier de données
- ↪ Coût d'une recherche dans l'index $<$ à celui d'une recherche dans le fichier principal
- Problème ! : maintenir l'ordre sur le fichier de donnée et sur l'index !

Index non-dense, Exercice, table *Cours*

CODE	Intitule	Responsable	nb_auditeur
glg102	SGBD Aspects Système	Asma Berriri	50
glg203	Complément Objet	Frederic Voisin	100
nfa114	Projets	Anastasia Bezerianos	30
smb114	Compilation	Frederic Voisin	40
nfe212	Intro base de données	Asma Berriri	45
nfe006	Programmation C++	Emmanuelle Frenoux	42
nfa003	Interfaces HM	Anastasia Bezerianos	37
nfe115	Concepts Objet et Java	Aurélien Max	38
nsy114	Vérif. and validation	Burkhart Wolff	72

CODE et *nb_auditeur* codés sur 2 octets chacun, *Intitule* et *Responsable* sur 60 o chacun. Le SGBD crée des blocs de 512 o et un pointeur fait 18 o (Supposant que entête d'un bloc contient uniquement pointeur)

- 1 Créez les blocs ordonnés sur l'attribut *Responsable*
- 2 Créez un index non dense sur ces blocs
- 3 Quel est la taille de l'index (en nbre de blocs) ?
- 4 Combien faut-il d'accès disques pour récupérer le cours de 'Frederic Voisin' ? (Supposons l'index en mémoire)



Index non-dense, Exercice *Cours*, Correction

les blocs, ordonnés sur *Responsable*

Bloc	CODE	Intitule	Responsable	nb_auditeur
B1	nfa003	Interfaces HM	Anastasia Bezerianos	37
	nfa114	Projets	Anastasia Bezerianos	30
	nfe212	Intro base de données	Asma Berriri	45
B2	glg102	SGBD Aspects Système	Asma Berriri	50
	nfe115	Concepts Objet et Java	Aurélien Max	38
	nsy114	Vérif. and validation	Burkhart Wolff	72
B3	nfe006	Programmation C++	Emmanuelle Frenoux	42
	glg203	Complément Objet	Frederic Voisin	100
	smb114	Compilation	Frederic Voisin	40

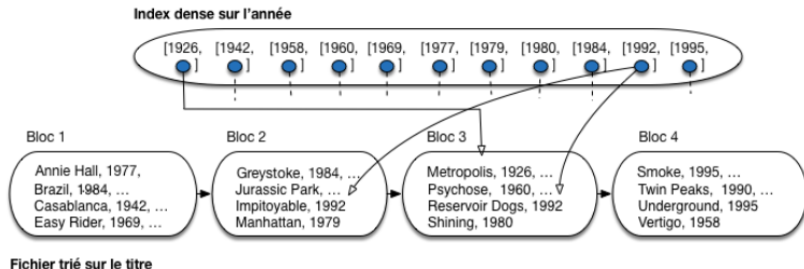
Index non dense

Anastasia Bezerianos	1
Asma Berriri	2
Emmanuelle Frenoux	3

- entrée d'index: $60 + 18 = 78$ o
- $\lfloor \frac{494}{78} \rfloor = 6$ entrées/bloc-index,
- Taille index: $\lceil \frac{3}{6} \rceil = 1$ bloc
- Cherche dans l'index la plus grande k' tel que $k' < \text{'Frederic Voisin'} \Rightarrow k' = \text{'Emmanuelle Frenoux'}$
 $\Rightarrow$ Bloc 3 et on cherche les articles de clé $k = \text{'Frederic Voisin'}$ dans Bloc 3 (et éventuellement dans les blocs données suivants) $\Rightarrow$ 1 lecture

- Fichier de données non trié
- On ne peut plus tirer parti de l'ordre des enregistrements pour introduire seulement dans l'index la valeur de clé du premier élément de chaque bloc
 - ⇒ Il faut donc baser l'index sur toutes les valeurs de clé existant dans le fichier, et les associer à l'adresse d'un enregistrement, et pas à l'adresse d'un bloc
- Toutes les valeurs de clé k sont représentées dans I
 - Volumineux
 - Temps de parcours lent

Index dense Exemple

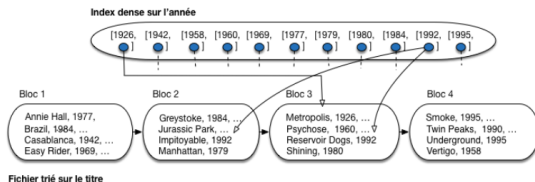


- Sur notre fichier de 1000000 films
 - Une année = 4 octets, une adresse = 8 octets
 - Taille de l'index : $1000000 * (4 + 8) = 12 \text{ Mo}$
- Encore 100 fois plus petit que le fichier de données

Index dense, recherche par clé

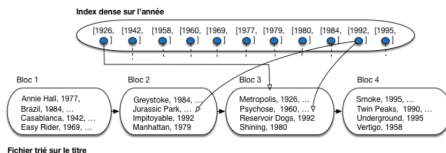
Si la **clé** n'est **pas unique**, il faut lire dans l'index toutes les adresses correspondant au critère de recherche

- Les films parus en 1992: suivre les adresses pointant respectivement sur Jurassic Park, Impitoyable et Reservoir Dogs
- Rien ne garantit que les films parus en 1992 sont situés dans le même bloc données: l'index est **non-plaçant**!!



Index dense, recherche par intervalle ([1950, 1979])

- 1 Recherche la 1ère valeur comprise dans l'intervalle: 1958;
- 2 Accès au bloc de données pour y prendre l'enregistrement Vertigo;
- 3 Parcourir la suite de l'index, en accédant à chaque fois à l'enreg correspondant dans le fichier de données, jusqu'à trouver une année > 1979: Psychose (3ème bloc), Easy Rider, Annie Hall (1er bloc) et Manhattan (2ème bloc)



- Pour trouver 5 enregs, on a accédé aux 4 blocs données, dans un ordre quelconque. **Moins efficace** que de parcourir les 4 blocs du fichier **en séquence** !
- Le coût est, dans le pire des cas, égale à r où r est le nbre d'enregistrements du résultat (soit 1 lecture de bloc de donnée par enreg)

Index dense, Exercice, table *Cours*

CODE	Intitule	Responsable	nb_auditeur
glg102	SGBD Aspects Système	Asma Berriri	50
glg203	Complément Objet	Frederic Voisin	100
nfa114	Projets	Anastasia Bezerianos	30
smb114	Compilation	Frederic Voisin	40
nfe212	Intro base de données	Asma Berriri	45
nfe006	Programmation C++	Emmanuelle Frenoux	42
nfa003	Interfaces HM	Anastasia Bezerianos	37
nfe115	Concepts Objet et Java	Aurélien Max	38
nsy114	Vérif. and validation	Burkhart Wolff	72

CODE et *nb_auditeur* codés sur 2 octets chacun, *Intitule* et *Responsable* sur 60 o chacun. Le SGBD crée des blocs de 512 o et un pointeur fait 18 o (Supposant que entête d'un bloc contient uniquement 1 pointeur)

- 1 Créez un index dense sur l'attribut *Responsable*. (Notez² ROWID: $x@y$)
- 2 Quel est la taille de l'index en nombre de blocs?
- 3 Combien faut-il d'accès disques pour récupérer les cours de Frédéric Voisin? (index en mémoire)

² x : adresse-bloc-sur-disque et y : position-dans-bloc

Index dense, Exercice Cours, Correction

Bloc	CODE	Intitule	Responsable	nb_auditeur
B1	glg102	SGBD Aspects Système	Asma Berriri	50
	glg203	Complément Objet	Frederic Voisin	100
	nfa114	Projets	Anastasia Bezerianos	30
B2	smb114	Compilation	Frederic Voisin	40
	nfe212	Intro base de données	Asma Berriri	45
	nfe006	Programmation C++	Emmanuelle Frenoux	42
B3	nfa003	Interfaces HM	Anastasia Bezerianos	37
	nfe115	Concepts Objet et Java	Aurélien Max	38
	nsy114	Vérif. and validation	Burkhart Wolff	72

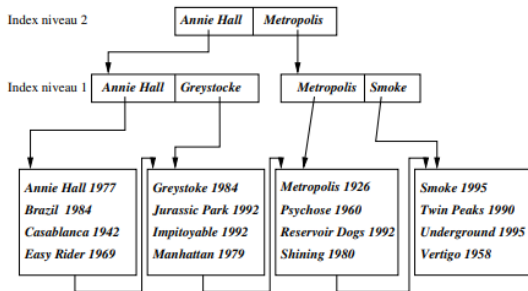
Index dense:

Anastasia Bezerianos	1@3
Anastasia Bezerianos	3@1
Asma Berriri	1@1
Asma Berriri	2@2
Aurélien Max	3@2
Burkhart Wolff	3@3
Emmanuelle Frenoux	2@3
Frederic Voisin	2@1
Frederic Voisin	1@2

- 6 clés différentes réparties sur 9 pointeurs.
- entrée d'index: $60 + 18 = 78$ o
- $\lfloor \frac{494}{78} \rfloor = 6$ entrées/bloc-index,
- Taille index: $\lceil \frac{9}{6} \rceil = 2$ blocs
- Recherche dans l'index tel que $k = \text{'Frederic Voisin'}$
retourne: $2@1 \Rightarrow$ lecture B2, et $1@2 \Rightarrow$
lecture B1 $\Rightarrow$ 2 lectures de blocs de données

Index multi-niveaux : on indexe l'index

- L'**index** est **trié** $\Rightarrow$ on peut l'indexer par un **2nd niveau non-dense**
- On obtient un index à plusieurs niveaux $\Rightarrow$ un arbre
 - Feuilles pointent vers fichier de données
 - Nœuds internes = index



Index multi-niveaux : on indexe l'index, 16 films

Hypothèse: 1 bloc index ne contient au plus que 4 entrées [année, ROWID]

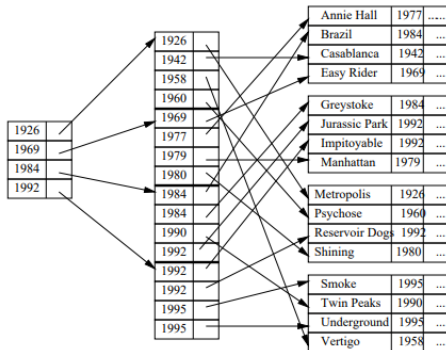
- L'index du 1er niveau (feuilles): dense

$$\left\lceil \frac{\text{nbre total enregistrements}}{\text{nbre enregistrements par bloc}} \right\rceil$$
$$= \left\lceil \frac{16}{4} \right\rceil = 4 \text{ blocs index}$$

- L'index du 2nd niveau: non-dense

⇒ construit sur la clé du 1er enregistrement de chaque bloc de l'index du 1er niveau

⇒ Il suffit donc de 1 bloc pour ce second niveau



Intérêt: pouvoir passer, dès le 2nd niveau, d'une structure dense à une structure non-dense

Index multi-niveaux : jusqu'où ?

Hypothèse: 1 bloc index ne contient au plus que 3 entrées [année, ROWID]

- L'index du 1er niveau (feuilles): dense

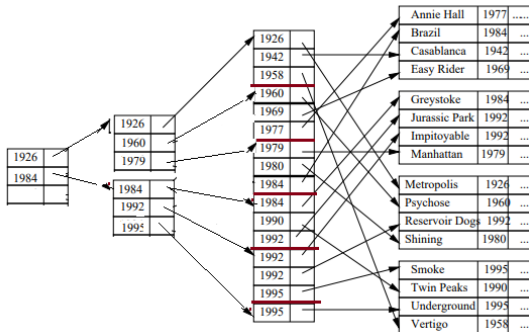
$$\left\lceil \frac{\text{nbre total enregistrements}}{\text{nbre enregistrements par bloc}} \right\rceil$$
$$= \left\lceil \frac{16}{3} \right\rceil = 6 \text{ blocs index}$$

- L'index du 2nd niveau: non-dense

$$\left\lceil \frac{\text{nbre blocs 1er niveau}}{\text{nbre enregistrements par bloc}} \right\rceil$$
$$= \left\lceil \frac{6}{3} \right\rceil = 2 \text{ blocs index}$$

- L'index du 3ème niveau: non-dense

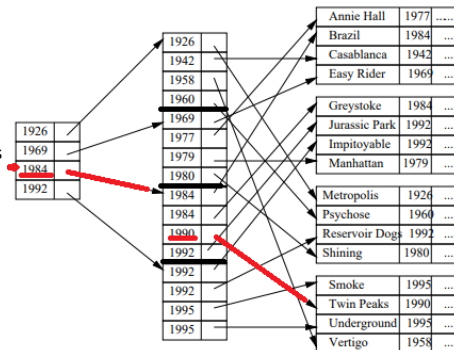
$$\left\lceil \frac{\text{nbre blocs 2er niveau}}{\text{nbre enregistrements par bloc}} \right\rceil$$
$$= \left\lceil \frac{2}{3} \right\rceil = 1 \text{ blocs index}$$



- Arrêt quand racine constitué d'un seul bloc
- Structure hiérarchique ; recherche de haut en bas
- Pour une recherche par clé, le coût est égal au nombre de niveaux de l'arbre

recherche le ou les films parus en 1990

- Partant du second niveau d'index, on sélectionne la troisième entrée correspondant à la clé 1984
- L'entrée suivante a en effet pour valeur 1992, et ne pointe donc que sur des films ultérieurs à cette date
- La troisième entrée mène au troisième bloc de l'index de premier niveau. On y trouve une entrée avec la valeur 1990 (il pourrait y en avoir plusieurs). Il reste à accéder à l'enregistrement.



Index composé

- Clé composée considérée comme une clé simple formée de la concaténation des attributs

- *create index on NomTable*($a_1, a_2, a_3, \dots, a_n$)

- Sélection par préfixe de la clé composée

- Il existe n préfixes : $(a_1), (a_1, a_2), \dots, (a_1, a_2, \dots, a_n)$

- Rq!! (a_2, a_3) n'est pas un préfixe

- Index composé utilisable pour une requête

Règle d'utilisation: **une seule traversée de l'index depuis la racine vers une feuille, suivie éventuellement d'un parcours latéral des feuilles**

↪ On appelle $(p_1, p_2, \dots, p_m)$ les attributs mentionnés dans le prédicat de sélection

↪ On détermine le plus grand préfixe de la clé composée : $(p_1, p_2, \dots, p_k)$

↪ Prédicat d'égalité pour tous les attributs p_1 à p_{k-1}

↪ Égalité, inégalité ou comparaison de préfixe pour le dernier attribut p_k

↪ Les prédicats p_{k+1} à p_m sont évalués par un filtre après l'accès à l'index.



Index composé: exemple

```
create index I1 on Personne(age, ville);
```

- Utilisable pour les requêtes :

```
Select * from Personne ...
```

A) `Where age = 18 and ville = 'Paris '`

B) `Where age = 18 and ville
like 'M%'`

C) `Where age >= 18`

D) `Where age >= 18 and ville =
'Paris '`

racine → 18 → première ville
commençant par M

puis parcours latéral des feuilles situées à
droite

racine → 18 → ville1 puis parcours latéral
des feuilles situées à droite

index seulement pour $\text{age} \geq 18$: racine
→ 18 → Paris puis parcours latéral des
feuilles situées à droite
puis filtre(ville='Paris')

- Inutilisable pour :

E) `Where ville= 'Paris '`



Structure d'index Récap

- Un index accélère des opérations de recherche portant sur la clé d'indexation ou sur un préfixe de la clé
- La structure repose sur le **tri des valeurs de clé indexées**
- On peut créer un index non-dense sur un fichier trié, dense sur un fichier non-trié
- Les index multi-niveaux s'appuient sur un **premier niveau dense**, puis sur des **niveaux supérieurs non-denses**
- Les index ont un coût, il faut les maintenir au fur et à mesure des insertions dans le fichier indexé
- Problème: difficulté de maintenir des fichiers triés sans dégradation des performances
- L'arbre-B+, étudié dans la section qui suit, à des performances équivalentes à celles des index en recherche, ajoute des algorithmes de réorganisation dynamique qui garantissent que la structure reste valide quelles que soient les séquences d'insertion et suppression subies par les données

L'arbre B: index hiérarchique

- Améliore l'efficacité des recherches
 - L'arbre est peu profond
 - chemin court de la racine vers une feuille
 - Toujours équilibré (Balanced tree en anglais)
 - Tous les chemins de la racine aux feuilles ont la même longueur
 - Suffisamment compact
 - Peut souvent tenir en mémoire
 - Un noeud est au moins à moitié rempli
- Coût d'accès
 - proportionnel à la longueur d'un chemin
 - identique quelle que soit la feuille atteinte
 - prévisible
- Avantage : Optimise les requêtes
 - permet d'estimer le coût d'accès, a priori, pour décider d'utiliser ou non un index
- Mesure du coût:
 - Nombre de noeuds (blocs index) lus / écrits
 - Nombre de blocs/pages de données lues / écrits

Définition

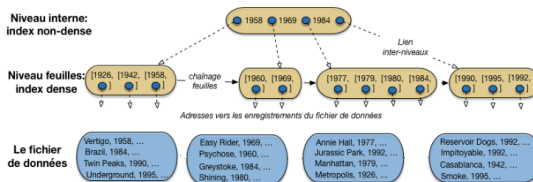
- ↪ L'ordre d'un arbre-B correspond au nombre minimal d'entrées (enregistrements $[k, ROWID]$) contenues dans chacun des noeuds
- ↪ Si un noeud peut contenir au maximum n entrées, alors l'ordre de l'arbre B est $k = \lfloor \frac{n}{2} \rfloor$

($\lfloor x \rfloor$ est le plus grand entier relatif inférieur ou égal à x)

- L'ordre d'un arbre dépend de la taille de la clé d'indexation (k) :
 - Soit $\|k\|$ la taille de la clé à indexer
 - $\|A\|$ la taille d'une adresse ($ROWID$)
 - $\|B\|$ l'espace utile d'un bloc-index (en excluant l'entête)
- ↪ Le nombre maximum d'entrées par bloc $n = \lfloor \frac{\|B\|}{\|A\| + \|k\|} \rfloor$
- ↪ L'ordre est $k = \lfloor \frac{n}{2} \rfloor$

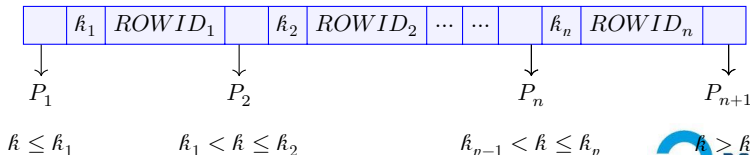
Arbre B d'ordre k: caractéristiques

- Si un noeud peut contenir au maximum n entrées, alors l'ordre de l'arbre est $k = \lfloor \frac{n}{2} \rfloor$
- Chaque **noeud** occupe un **bloc index** et contient entre k et $(2 * k)$ entrées (enregistrements $[k, ROWID]$)
 - Seule exception: la racine contient entre 1 et $(2 * k)$ entrées
- Les entrées d'un noeud sont **triés** sur la clé k
- Chaque noeud interne avec n entrées a $n + 1$ fils
 - Chaque noeud interne est un index non-dense pour ses fils/descendants
- Le dernier niveau est celui des feuilles de l'arbre. Il constitue un index dense alors que les niveaux supérieurs sont non-denses



Structure du noeud d'un arbre B

- n entrées, en **ordre croissant** des clés ($k \leq n \leq 2k$)
 - $k_1 \leq k_2 \leq \dots \leq k_n$: clés
 - $ROWID_1, ROWID_2, \dots, ROWID_n$: l'adresse physique de l'enregistrement de clé k_i
- $(n + 1)$ pointeurs vers les noeuds fils dans l'index: $P_1, \dots, P_{n+1}$
 - Toutes les clés k dans le sous-arbre pointé par P_i respectent la condition $k_{i-1} < k \leq k_i$ ($2 \leq i \leq n$)
 - Pour $i = 1 \Rightarrow k \leq k_1$, pour $i = n + 1 \Rightarrow k_n < k$
 - Les n clés déterminent $(n + 1)$ intervalles, chaque pointeur P correspond à l'un de ces intervalles



Indexons un million de films sur l'année

- Admettons qu'une entrée d'index occupe 12 octets, soit 8 octets pour l'adresse (*ROWID*), et 4 octets pour la clé k (ex. année du film)
 - Chaque bloc index est de 4096 octets. On place donc $\lfloor \frac{4096}{(8+4)} \rfloor = 341$ entrées (au maximum) dans un bloc index
 - l'ordre de cet index est $k = \lfloor \frac{341}{2} \rfloor = 170$
- ↪ Comme le **niveau des feuilles** de l'arbre B est **dense**, il faut $\lceil \frac{1000000}{341} \rceil = 2932$ blocs pour le niveau des feuilles
- ↪ Le **deuxième niveau** est **non-dense**. Il comprend **autant d'entrées que de blocs (du 1er niveau) à indexer**, soit 2932 entrées. Il faut donc $\lceil \frac{2932}{341} \rceil = 8$ blocs (au mieux)
- ↪ Finalement, un **troisième niveau non-dense**, constitué d'un **bloc** avec 8 entrées suffit pour compléter l'index

Hypothèse: le fichier de données stocke séquentiellement les enregistrements dans l'ordre de leur création et donc indépendamment de tout ordre lexicographique ou numérique sur l'un des attributs

Capacité, hauteur d'un arbre B

- avec 1 niveau d'index (racine seulement) on peut référencer 341 films
- avec 2 niveaux, la racine indexe 341 blocs d'index, référençant chacun 341 films, soit $341^2 = 1162816$ films
- avec 3 niveaux on indexe $341^3 = 39651821$ films
- enfin avec 4 niveaux on indexe plus de 1 milliard de films

- ⇒ Croissance exponentielle, du nombre d'enregistrements indexés en fonction du nombre de niveaux, et croissance très faible -logarithmique- du nombre de niveaux en fonction du nombre d'enregistrements
- ⇒ Supposons que les clés sont uniques. On note $||T||$ la cardinalité d'une table T (nombre d'enregistrements), et k l'ordre de l'arbre B. Alors la hauteur h de l'arbre est

$$h = \lceil \log_k (||T||) \rceil$$

Recherche des articles de clé k : On part de la racine de l'arbre

↪ les clés dans l'arbre sont distinctes

- Soit $k_1, k_2, \dots, k_n$, les clés du noeud courant
- Si k est parmi ces clés ($k = k_i$) $\Rightarrow$ **résultat** = l'article récupéré à l'aide de $ROWID_i$
- Sinon, si le noeud courant est une feuille $\Rightarrow$ **résultat** = aucun article
- Sinon on choisit l'intervalle $k_{i-1} < k < k_i$ (ou $k < k_1$ ou $k > k_n$) dans lequel se trouve k et on continue récursivement avec le noeud fils indiqué par P_i

↪ Si les clés ne sont pas distinctes:

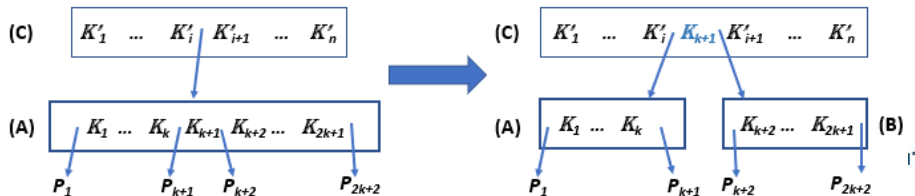
- Une clé peut dans ce cas apparaître plusieurs fois dans le même noeud et/ou dans des noeuds différents
- Si k est parmi les clés du noeud courant ($k = k_i$) $\Rightarrow$ en plus de l'article récupéré à l'aide de $ROWID_i$, on cherche aussi dans les fils à gauche (P_i) et à droite (P_{i+1}) de la clé

Technique similaire lorsqu'on cherche un intervalle de clés

Arbre B d'ordre k : Insertion

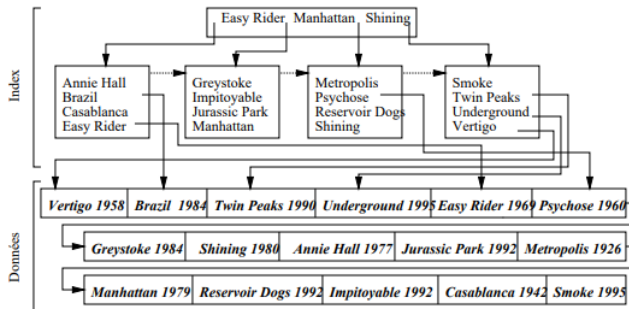
On considère des clés distinctes et on insère une nouvelle clé $\mathcal{K}$:

- ❶ On cherche la feuille de l'arbre où doit se faire l'insertion (recherche de $\mathcal{K}$)
 - On insère $\mathcal{K}$ à sa place dans la feuille (en gardant l'ordre croissant des clés)
- ❷ Si le noeud (A) déborde suite à l'insertion ($\mathcal{K}$ est la $(2 * k + 1)^{\text{ième}}$ clé)
 - ❶ On crée un nouveau noeud (B)
 - ❷ On garde les k premières clés dans (A) et on met les k dernières dans (B)
 - ❸ La clé du milieu (la $(k + 1)^{\text{ième}}$) est insérée dans le noeud parent (C)
 - À gauche de cette clé reste le noeud (A), à droite se trouvera le noeud (B)
 - ❹ Si le noeud (C) déborde suite à cette insertion, on continue selon la même méthode,
 - ❺ Si la racine déborde, la clé du milieu formera une nouvelle racine à une seule clé



- Arbre B+: variante de l'arbre B qui simplifie la recherche par intervalles, en gardant les avantages de l'arbre B
- Principes Arbre B+
 - Toutes les clés de l'arbre se retrouvent dans les feuilles
 - Seules les feuilles contiennent les infos sur les enregistrements de données
 - Les feuilles sont chaînées entre elles
 - Les clés des noeuds internes servent à guider la recherche de feuilles
 - Redondance: les clés des noeuds internes se retrouvent aussi dans les feuilles
 - Les feuilles contiennent les adresses des enregistrements de données
- Remarques (en pratique)
 - Les noeuds internes stockent seulement des clés k_i (pas de $ROWID_i$)
 - Les feuilles n'ont pas besoin de stocker des pointeurs P_i

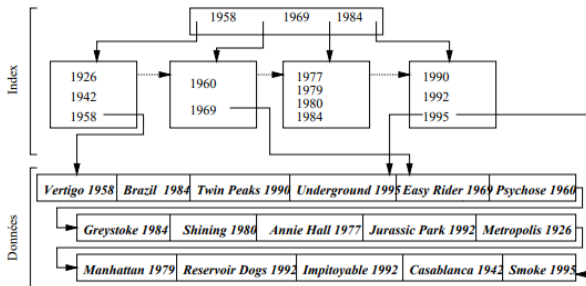
Exemple: Un arbre-B+ indexant 16 films, clé unique, hypothèse: $n = 4$ entrées par noeud



- Au sein d'un même bloc index, les titres sont triés par ordre lexicographique!
- Les blocs index sont chaînés entre eux: structure arborescente à deux niveaux ici
 - 1er niveau: 1 bloc: racine: 3 titres et 4 chaînages vers 4 blocs du 2ème niveau
 - bloc à gauche de Easy Rider: titres $\leq$ à Easy Rider
 - bloc entre Easy Rider et Manhattan: titres $>$ à Easy Rider et $\leq$ à Manhattan
 - Et ainsi de suite: le dernier bloc contient des titres $>$ à Shining
- Feuilles sont liées entre elles en suivant l'ordre lexicographique des valeurs qu'elles contiennent: chaînage (en pointillés) pour recherches par intervalle

Exemple: Un arbre-B+ indexant 16 films, clé non unique, hypothèse: $n = 4$ entrées par noeud

- ⇒ Sous-arbre référencé par l'adresse située à gauche de 1958: que des films parus avant (au sens large) 1958;
- ⇒ Sous-arbre référencé par l'adresse entre 1958 et 1969: que des films parus après (au sens strict) 1958 et avant (au sens large) 1969;
- ⇒ Sous-arbre référencé par l'adresse entre 1969 et 1984: que des films parus après (au sens strict) 1969 et avant (au sens large) 1984;
- ⇒ Dernière adresse référence un sous-arbre contenant tous les films parus strictement après 1984.



- Cet index permet d'optimiser des requêtes utilisant l'année comme critère de recherche
- Cet index n'est pas construit sur des attributs formant une clé unique $\Rightarrow$ plusieurs adresses associés à une même clé d'indexation (1995 est associée à deux films, Smoke et Underground)

Opérations sur l'arbre B+

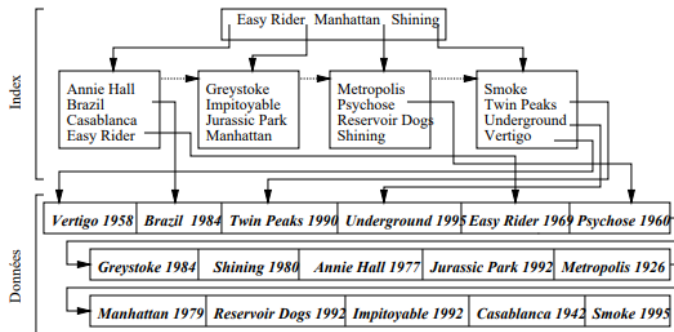
L'arbre-B+ supporte des opérations de recherche par **clé**, par **préfixe** de la clé et par **intervalle**

- Recherche par clé
 - Similaire à l'arbre B, mais on descend toujours jusqu'aux feuilles
- Recherche par intervalle
 - On cherche la **clé de début d'intervalle**, ensuite on **parcourt séquentiellement les feuilles** jusqu'à la fin de l'intervalle
- Insertion
 - En cas de **débordement de la feuille**, la clé **médiane** qui remonte vers le nœud parent est aussi **gardée** dans l'une des deux feuilles résultantes (**feuille gauche**)

- ↪ Hauteur de l'arbre: $\approx$ celle d'un arbre B
- ↪ Recherche de clé: $\approx (h_{B+} + 1)$ lectures (les adresses d'articles sont dans les feuilles)
- ↪ Recherche par intervalle: $\approx (h_{B+} + \text{nbre feuilles avec clés dans l'intervalle})$

Arbre B+: Recherche par clé

```
SELECT *  
FROM Film  
WHERE titre = '  
        Impitoyable'
```



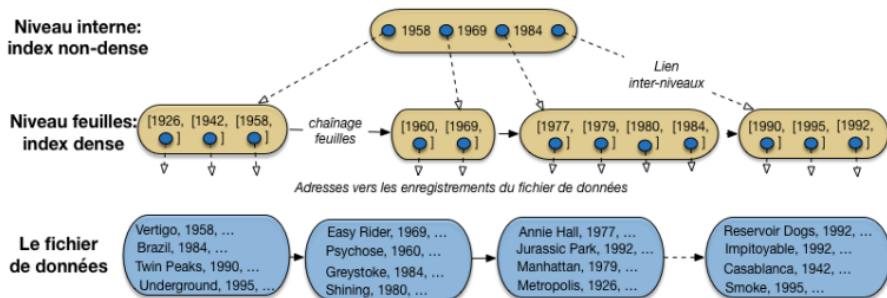
- Absence d'index: parcours séquentiel du fichier données $\Rightarrow$ lecture de 13 films avant de trouver Impitoyable $\Rightarrow$ 3 lectures (blocs données)
- Avec index: $\Rightarrow$ 3 lectures sont suffisantes (2 blocs index et 1 données)
 - lecture racine de l'arbre : Impitoyable étant situé dans l'ordre lexicographique entre Easy Rider et Manhattan, suivre le chaînage situé entre ces deux titres ;
 - lecture du bloc feuille dans lequel on trouve le titre Impitoyable associé à l'adresse de l'enregistrement dans le fichier des données ;
 - lecture l'enregistrement de donnée
- Généralement, le nombre d'accès disques nécessaires pour une recherche par clé = nombre de niveaux de l'arbre, plus une lecture pour accéder au fichier de données

Arbre B+: Recherche par clé

- Fichier contenant un million de films
- Un enregistrement données = 1200 octets
- Un bloc données = 4 096 octets
- $\lfloor \frac{4096}{1200} \rfloor = 3$ enregistrements par bloc de données
- environ $\lceil \frac{1000000}{3} \rceil = 300000$ blocs données (Pour simplifier)
- Une entrée d'index occupe 28 octets $\Rightarrow$ On place donc $\lfloor \frac{4096}{28} \rfloor = 146$ entrées (au maximum) dans un bloc index
 - $\hookrightarrow$ 1er niveau: Il faut $\lceil \frac{1000000}{146} \rceil = 6850$ blocs
 - $\hookrightarrow$ 2ème niveau comprend 6850 entrées, une pour chaque bloc du 1er niveau. Il faut donc $\lceil \frac{6850}{146} \rceil = 47$ blocs
 - $\hookrightarrow$ 3ème niveau: 1 bloc suffit avec 47 entrées
- Recherche par clé
 - Sans index: 30 000 accès disques (parcours séquentiel: lecture de 30 000 blocs données)
 - Avec index: 4 accès disques (3 blocs index, 1 bloc données)

Recherche par intervalle

```
select *  
from Film  
where annee between 1960 AND 1975
```



- Recherche par clé pour l'année 60. Accède à la seconde feuille, on trouve la valeur 1960 associée à l'adresse du film correspondant (Psychose) dans le fichier des données
- Parcours les feuilles de l'arbre en suivant le chaînage, jusqu'à l'année 1975
- à chaque fois on lit l'enregistrement

Recherche par préfixe

Exemple :

```
SELECT *  
FROM Film  
WHERE titre LIKE 'M%'
```

Revient à une recherche par intervalle

```
SELECT *  
FROM Film  
WHERE titre BETWEEN 'MAAAAAA...' AND 'MZZZZZZ...'
```

Attention! Contre-exemple : recherche par **suffixe**: Pas possible!

```
SELECT *  
FROM Film  
WHERE titre LIKE '%e'
```

Ici **index inutilisable**

Arbre B+ d'ordre k : Insertion

On insère une nouvelle clé $\mathcal{K}$:

- ❶ En cherchant dans l'arbre, trouver la feuille où l'élément pourrait être inséré.
- ❷ Si le noeud contient moins de valeurs que le nombre maximum autorisé par noeud ($2 * k$), alors ajouter l'élément de manière *ordonnée*.
- ❸ Sinon, la feuille est alors éclatée :
 - ❶ L'élément médiant (le $(k + 1)^{\text{ième}}$ élément) est choisi (nouveau père du sous arbre) parmi les éléments présents et le nouveau (il y a $2 * k + 1$ éléments).
 - ❷ Les valeurs *inférieures ou égal* au médiant deviennent fils gauche, et les valeurs *supérieures* deviennent fils droit.
 - ❸ L'élément médiant (père du sous-arbre) est ajouté au noeud parent (celui qui pointait sur le noeud courant). Un nouvel éclatement peut alors en résulter (*dans ce cas, seul les valeurs inférieures deviennent fils gauche, et non inférieures ou égales*).

Construction Arbre B+/ 1er exemple: Fichier *Cours*

- Table *Cours* (*CODE*, *Intitule*, *Responsable*, *nb_auditeur*)
- Soit le fichier contenant les données de cette table. Voici un extrait

Bloc	CODE	Intitule	Responsable	nb_auditeur
B1	glg102	SGBD Aspects Système	Asma Berriri	50
	glg203	Complément Objet	Frederic Voisin	100
	nfa114	Projets	Anastasia Bezerianos	30
B2	smb114	Compilation	Frederic Voisin	40
	nfe212	Intro base de données	Asma Berriri	45
	nfe006	Programmation C++	Emmanuelle Frenoux	42
B3	nfa003	Interfaces HM	Anastasia Bezerianos	37
	nfe115	Concepts Objet et Java	Aurélien Max	38
	nsy114	Vérif. and validation	Burkhart Wolff	72

- On souhaite accéder :
 - Au Responsable, Intitule d'un cours grâce à son CODE
 - Aux intitulés des cours d'un responsable
 - Aux nombre de cours ayant un certain nombre d'auditeurs

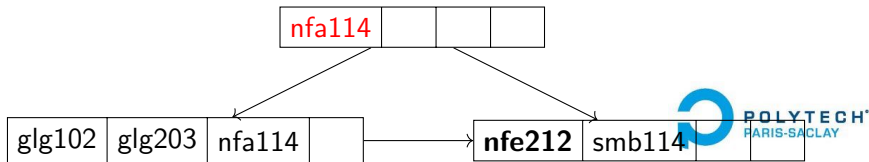
⇒ Créons un index, arbre B+ d'ordre 2, sur l'attribut *CODE*

Construction arbre B+ (Exemple *Cours*) (1)

- Ordre $k = 2 \Rightarrow$ Taille d'un noeud = $2 * 2 = 4$ entrées au plus
- Insérer ces éléments dans cet ordre:
glg102, glg203, nfa114, smb114, nfe212, nfe006, nfa003, nfe115, nsy114, rsx201

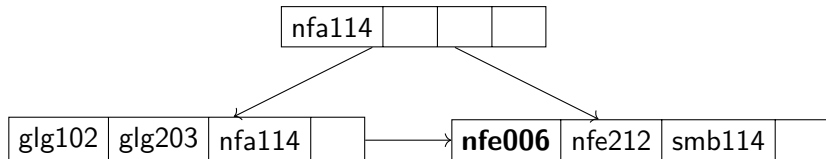
glg102			
glg102	glg203		
glg102	glg203	nfa114	
glg102	glg203	nfa114	smb114

glg102	glg203	nfa114	nfe212	smb114
--------	--------	--------	--------	--------

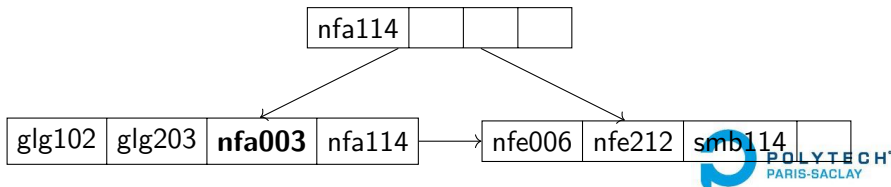


Construction arbre B+ (Exemple *Cours*) (2)

*glg102, glg203, nfa114, smb114, nfe212, **nfe006**, nfa003, nfe115, nsy114, rx201*

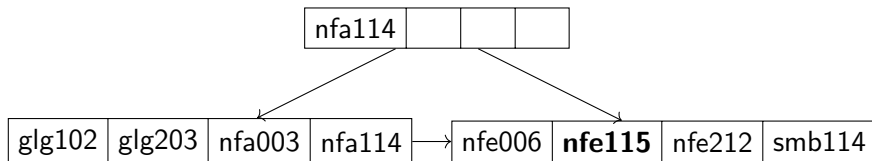


*glg102, glg203, nfa114, smb114, nfe212, nfe006, **nfa003**, nfe115, nsy114, rx201*

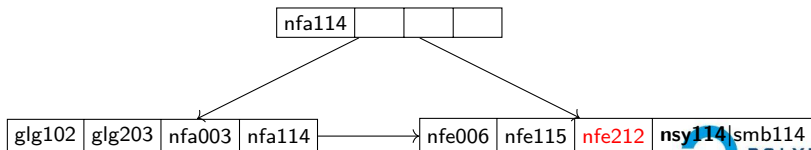


Construction arbre B+ (Exemple *Cours*) (3)

*glg102, glg203, nfa114, smb114, nfe212, nfe006, nfa003, **nfe115**, nsy114, rsx201*

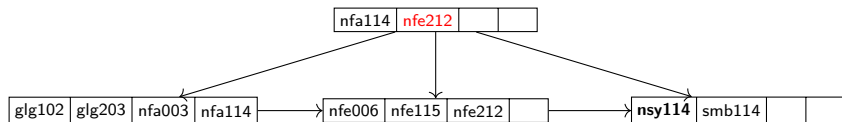


*glg102, glg203, nfa114, smb114, nfe212, nfe006, nfa003, nfe115, **nsy114**, rsx201*

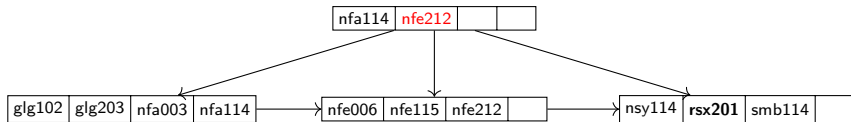


Construction arbre B+ (Exemple *Cours*) (4)

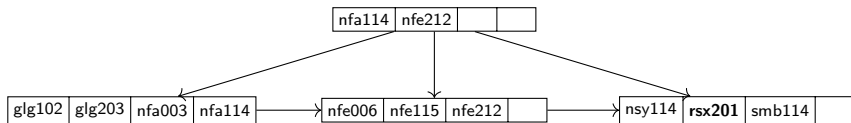
*glg102, glg203, nfa114, smb114, nfe212, nfe006, nfa003, nfe115, **nsy114**,
rsx201*



*glg102, glg203, nfa114, smb114, nfe212, nfe006, nfa003, nfe115, nsy114,
rsx201*



Construction arbre B+ (Exemple *Cours*) (5)

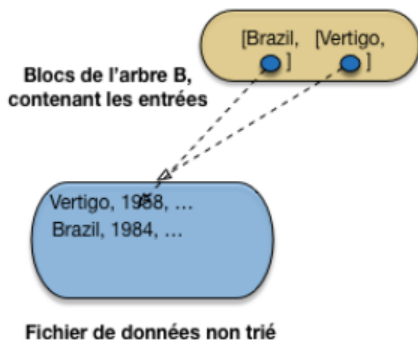


Exercice : continuer à insérer *nfa025*, *nfe214*, *nsy012*, *nfe207*, *nfe102*

Construction arbre B+ (Exemple *Film*)

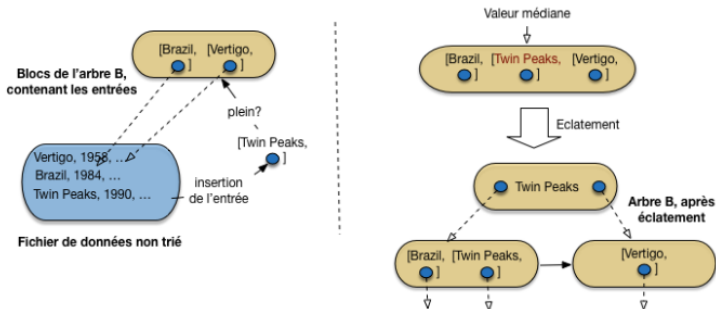
- Arbre B+ sur le *titre* des films: supposons au plus 2 entrées par bloc index
- Les enregistrements données sont placés les uns après les autres dans l'ordre de leur insertion, dans une structure dite séquentielle -blocs de données- (supposons 4 enregistrements par bloc de données)

Situation initiale : Début de la construction, 2 films seulement



Insertion de 'Twin Peaks'

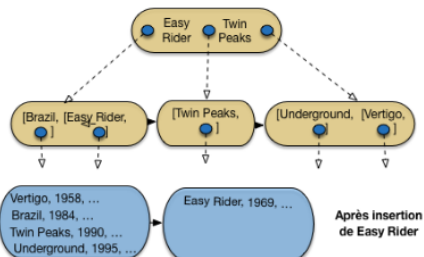
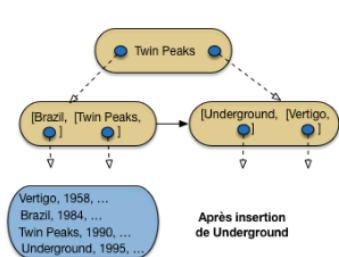
- L'enregistrement vient se mettre à la suite des autres dans le fichier de données
- Il faut ajouter un nouveau bloc à l'index: **une procédure d'éclatement**



- les entrées du bloc index plein sont triées sur la valeur de la clé
- l'entrée correspondant à la valeur médiane se place dans un nouveau bloc, au niveau sup
- les entrées $\leq$ à la valeur médiane sont dans un bloc à gauche; les entrées $>$ à la valeur médiane sont dans un bloc à droite

Insertions de 'Underground', puis 'Easy Rider'

- Les enregistrements sont placés séquentiellement dans le fichier de données
- Pour les entrées d'index, on doit déterminer dans quelle feuille on insère. Pour cela on part de la racine, et on suit les adresses comme si on recherchait un enregistrement pour la valeur de clé à insérer

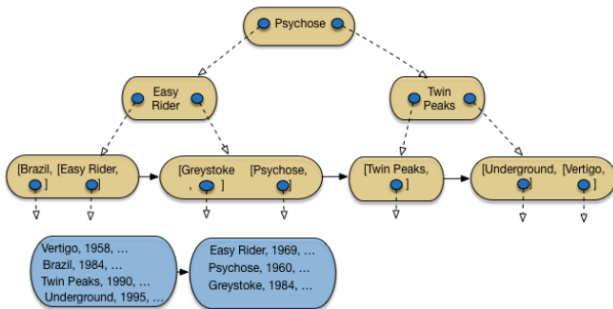


- Underground étant supérieur à Twin Peaks, va à droite, et Easy Rider va à gauche
- Easy Rider doit aller dans la feuille de gauche, qui devient trop pleine. Un éclatement a lieu, l'entrée correspondant à la valeur médiane (Easy Rider) est transmise au niveau supérieur qui indexe donc les trois blocs de feuilles

Insertion de 'Psychose' puis 'Greystoke': Ajout d'un niveau

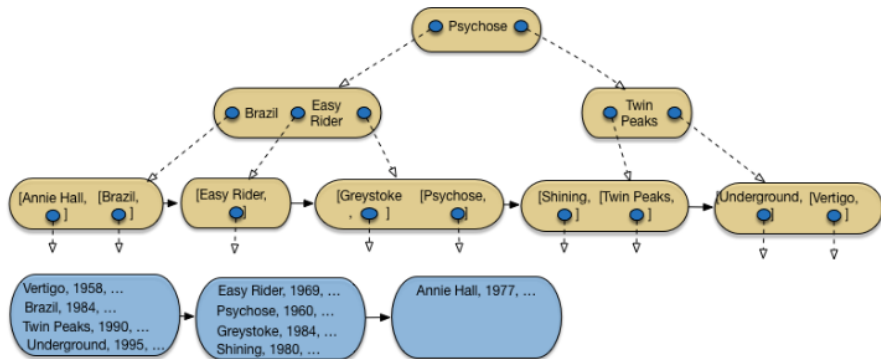
→ Tous deux vont dans le bloc contenant Twin Peaks ⇒ un débordement. La valeur médiane, Psychose, doit donc être insérée dans la racine

→ L'éclatement de la racine entraîne l'ajout d'un niveau

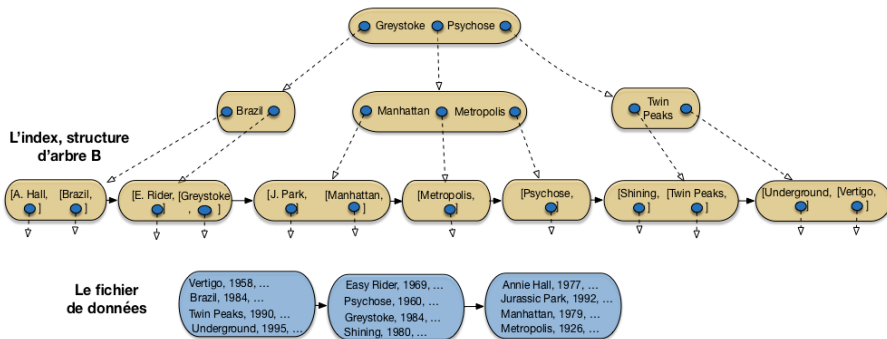


- ① Trouver la feuille où l'élément pourrait être inséré
- ② Si le noeud contient moins d'éléments que $(2 * k)$, alors ajouter l'élément de manière ordonnée
- ③ Sinon, éclatement :
 - ① Le médiant est le $(k + 1)^{\text{ième}}$ élément
 - ② Les valeurs $\leq$ au médiant deviennent fils gauche, et les valeurs $>$ deviennent fils droit.
 - ③ Le médiant est ajouté au noeud parent. Un nouvel éclatement peut alors en résulter (dans ce cas, seul les valeurs $<$ deviennent fils gauche, et non $\leq$).

Insertions de 'Shining' et 'Annie Hall'



Insertions de 'Jurassic Park', 'Manhattan' et 'Metropolis'



Je vous laisse compléter avec les 4 films restant, soit Reservoir Dogs, Impitoyable, Casablanca et Smoke

Arbre B+ : Propriétés importantes

- Chaque bloc (noeud) de l'arbre B+ (sauf la racine) est au moins à moitié plein. Obtenue par construction: un éclatement n'intervient que quand un bloc est plein, cet éclatement répartit les entrées en deux sous-ensembles de taille égale, et les deux blocs résultant d'un éclatement sont donc à moitié plein, avant de recevoir de nouvelles entrées avec les insertions ultérieures. On constate en pratique que le taux de remplissage est d'environ 70%
- Aspect dynamique de la construction: Une insertion dans un arbre B+ peut déclencher des réorganisations qui restent locales, n'affectent qu'une partie minimale de la structure globale

Création d'un arbre B+

Sur la clé primaire

```
create table Film (titre varchar(30) not null ,  
    ... ,  
    primary key (titre)  
);
```

Sur n'importe quel attribut (non unique, ex. sur l'année)

```
create index filmAnnee on Film (annee)
```

- Le SGBD synchronise le contenu de la table et celui de l'index
- La création de l'index sur la clé primaire (et parfois sur la clé étrangère) est très utile pour vérifier la satisfaction des contraintes de clé et pour le calcul des jointures
- Quand un arbre-B+ est créé sur une table, soit directement par la commande *CREATE INDEX*, soit indirectement avec l'option *PRIMARY KEY*, le SGBD effectue automatiquement les opérations nécessaires au maintien de la structure : insertions, destructions, mises à jour



Propriétés de l'arbre B+

- L'arbre B+ est une structure arborescente :
 - équilibré: tous les chemins de la racine vers les feuilles ont la même longueur;
 - chaque noeud (sauf la racine) est un bloc occupé au moins à 50% par des entrées de l'index;
 - une recherche s'effectue par une traversée en profondeur de l'arbre, de la racine vers les feuilles;
- Le coût des opérations est **logarithmique** dans la taille des données, alors qu'une recherche sans index est linéaire
- Il est (presque) parfait !
 - On a très rarement besoin de plus de trois niveaux
 - Le coût d'une recherche par clé est le nombre de niveaux + 1
 - Supporte les **recherches par clé**, par **intervalle**, par **préfixe**
 - Dynamique
- On peut juste lui reprocher d'occuper de la place